

Unit-04

Entity Framework and LINQ

4.1. Introduction to Entity Framework

Entity Framework (EF) is an Object-Relational Mapping (ORM) framework for .NET applications. It simplifies database operations by allowing developers to interact with a database using objects in C# or other .NET languages. EF eliminates the need for writing SQL queries directly and instead allows developers to use LINQ (Language Integrated Query) to query, insert, update, and delete data.

Concepts of Entity Framework:

1. ORM (Object-Relational Mapping):

- EF acts as a bridge between the object-oriented model and the relational database, converting database rows into C# objects and vice versa.

2. DbContext:

- This is the main class in EF that represents the session between the application and the database. It handles the operations related to data retrieval, persistence, and change tracking.

3. Entities:

- An entity represents a table in a database. In EF, entities are typically represented as C# classes, where each class corresponds to a table in the database.

4. DbSet:

- A `DbSet<TEntity>` is a collection of entities that can be queried from the database and that can be saved to the database. A `DbSet` is used to interact with entities in the database.

5. Code First, Database First, and Model First:

- **Code First:** The developer writes C# classes first, and EF generates the database schema based on those classes.
- **Database First:** The database schema is created first, and EF generates C# classes to map to the existing database.
- **Model First:** The developer creates an Entity Data Model (EDM) first, and EF generates both the classes and the database schema.

6. LINQ to Entities:

- Entity Framework allows developers to write queries using LINQ, providing a strongly-typed and powerful query mechanism to interact with data.

4.2. Understanding and Implementing Database First, Code First, and Model First Approaches

Entity Framework (EF) offers three approaches to handle the database schema and entity mappings: **Database First**, **Code First**, and **Model First**. Each approach has its specific use cases and workflows.

1. Database First Approach

The Database First approach in Entity Framework is used when the database schema is already designed or pre-existing. It allows developers to generate entity classes and a DbContext directly from the database structure.

Implementation Steps:

1. Create or Use an Existing Database:

- Ensure you have a functional database with the required schema and data.

2. Create a New Project:

- Open Visual Studio.
- Create a new project (e.g., a Console App or Web App).

3. Add an ADO.NET Entity Data Model:

- Right-click on the project in **Solution Explorer**.
- Select **Add > New Item**.
- Choose **ADO.NET Entity Data Model** and give it a name.

4. Choose Database First:

- In the wizard, select **EF Designer from Database**.

5. Connect to the Database:

- Provide the database connection details.
- Select the tables, views, and stored procedures you want to include.

6. Generate the Model:

- The wizard creates an Entity Data Model (EDM) with a .edmx file.

- It includes entity classes and a DbContext for data operations.

7. Use the Generated Classes:

- The generated classes represent your database tables.
- Interact with the database using LINQ queries via the DbContext.

2. Code First Approach

The Code First approach in Entity Framework allows developers to define the database structure directly in C# code using classes and properties. EF uses these classes to generate the database schema dynamically, providing developers with full control over the data model and schema evolution.

Steps to Implement Code First

1. Define Entity Classes

Create C# classes to represent your database tables and their properties.

```
public class Product
{
    public int ProductID { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

2. Create a DbContext

The DbContext class manages database operations and defines DbSet properties for each entity.

```
public class MyDbContext : DbContext
{
    public DbSet<Product> Products { get; set; }
}
```

3. Configure Connection String

Define the database connection in your application configuration file (appsettings.json or app.config).

```
"ConnectionStrings": {  
    "MyDbContext":  
    "Server=(localdb)\\mssqllocaldb;Database=MyDatabase;Trusted_Connection=True;"  
}
```

4. Enable Migrations

Use EF migrations to create and update the database schema.

5. Perform CRUD Operations

Use LINQ to query and modify data in the database.

```
using (var context = new MyDbContext())  
{  
    context.Products.Add(new Product { Name = "Laptop", Price = 1000 });  
    context.SaveChanges();  
    var products = context.Products.ToList();  
    var product = products.FirstOrDefault();  
    if (product != null)  
    {  
        product.Price = 1200;  
        context.SaveChanges();  
    }  
    context.Products.Remove(product);  
    context.SaveChanges();  
}
```

3. Model First Approach

The Model First approach in Entity Framework allows developers to design the database schema visually using the Entity Data Model Designer. From the model, EF generates both the database schema and the corresponding C# classes.

Using DbContext to Connect and Interact with the Database

Step 1: Install Entity Framework

Install-Package EntityFramework

Step 2: Define the Entity Class

This represents a table in your database.

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

Step 3: Define the DbContext

This class acts as the bridge between application and the database.

```
using System.Data.Entity;

public class MyDbContext : DbContext
{
    public MyDbContext() : base("name=MyDbConnectionString")
    {
    }

    public DbSet<Product> Products { get; set; }
}
```

Step 4: Configure the Connection String

Add the connection string to the App.config or Web.config file under <connectionStrings>.

```
<configuration>
  <connectionStrings>
    <add name="MyDbConnectionString"
        connectionString="Server=YOUR_SERVER_NAME;Database=YourDatabaseName;Trusted_
        Connection=True;"
        providerName="System.Data.SqlClient" />
  </connectionStrings>
</configuration>
```

Step 5: Perform Database Operations

Use the MyDbContext class to interact with the database.

```
using System;

using System.Linq;

class Program
{
    static void Main(string[] args)
    {
        using (var context = new MyDbContext())
        {
            var product = new Product { Name = "Laptop", Price = 1200.50M };
            context.Products.Add(product);
            context.SaveChanges();

            Console.WriteLine("Product added successfully!");

            var products = context.Products.ToList();
            Console.WriteLine("Products in Database:");
            foreach (var p in products)
            {
                Console.WriteLine($"Id: {p.Id}, Name: {p.Name}, Price: {p.Price}");
            }

            var productToUpdate = context.Products.FirstOrDefault();
            if (productToUpdate != null)
            {
                productToUpdate.Price = 1300.00M;
                context.SaveChanges();

                Console.WriteLine("Product updated successfully!");
            }

            var productToDelete = context.Products.FirstOrDefault();
```

```
        if (productToDelete != null)
        {
            context.Products.Remove(productToDelete);
            context.SaveChanges();
            Console.WriteLine("Product deleted successfully!");
        }
    }
}
```

4.3. LINQ and Its Implementation

LINQ (Language Integrated Query) is a powerful feature in C# that provides a consistent and readable way to query and manipulate data from various sources such as collections, databases, XML, and more. LINQ integrates query capabilities directly into the C# language, allowing you to work with data in a declarative style.

Types of LINQ

1. **LINQ to Objects:** Queries in-memory data like arrays, lists, and collections.
2. **LINQ to SQL:** Queries relational databases using SQL-like syntax.
3. **LINQ to XML:** Queries and manipulates XML data.
4. **LINQ to Entities:** Works with Entity Framework to query databases.
5. **LINQ to DataSet:** Queries DataSet and DataTable.

LINQ Methods

1. **.Where()** - Filters elements based on a condition.
2. **.Select()** - Projects each element into a new form.
3. **.OrderBy()** and **.OrderByDescending()** - Orders elements.
4. **.GroupBy()** - Groups elements by a key.
5. **.Join()** - Combines two sequences based on a key.
6. **.Aggregate()** - Performs a custom aggregation operation.
7. **.First()** and **.FirstOrDefault()** - Returns the first element.
8. **.Any()** - Checks if any element satisfies a condition.

LINQ to SQL

LINQ to SQL enables querying databases directly using LINQ syntax.

```
using System;
using System.Linq;
class Program
{
    static void Main(string[] args)
    {
        using (var context = new MyDbContext())
        {
            var expensiveProducts = from p in context.Products
                                   where p.Price > 1000
                                   select p;

            Console.WriteLine("Expensive Products:");
            foreach (var product in expensiveProducts)
            {
                Console.WriteLine($"Name: {product.Name}, Price: {product.Price}");
            }

            var cheapProducts = context.Products.Where(p => p.Price <= 1000).ToList();
            Console.WriteLine("Cheap Products:");
            foreach (var product in cheapProducts)
            {
                Console.WriteLine($"Name: {product.Name}, Price: {product.Price}")
            }
        }
    }
}
```